DATA STRUCTURES AND ALGORITHMS-3 (25CS2103E)

Plagiarism Detection in Academic Submissions

Team Members:
V.Hari Priya : 2520080015
M.Sai Harika : 2520080068
Anirudh Ganesh : 2520080058

Under the Esteemed Guidance of : Dyuti Banerjee
AI & DS

Made with GAMMA

Contents

1. Abstract, Introduction, Literature Survey
2. Problem Statement and Objectives
3. Proposed Methodology / System Architecture
4. System Design / Workflow
5. Final Implementation, Implementation Modules
1. Algorithm / Method Used, Source Code / Key Code Snippets
2. Testing and Validation
3. Experimental Results / Output Screenshots
4. Performance Analysis, Discussion
5. Conclusion, Future Scope, References

Made with GAMMA

Abstract

Academic plagiarism is growing as copied text is easy to find, paraphrase and submit, and manual checking does not scale.

The project builds a detector that compares student submissions against each other and against a reference corpus.

PIPELINE

Ingest documents, normalize text, break it into shingles or n-grams, fingerprint with hashing, and match similar passages.

OUTPUT

A similarity score and highlighted matching passages for each submission.

Made with GAMMA

Introduction

- Plagiarism is presenting someone else's work as your own, from verbatim copying to paraphrasing and structural copying.
- Detection is a string and text matching problem, so it depends on data structures such as hash tables, tries and sets.
- Common techniques: n-gram shingling, Jaccard similarity, Rabin-Karp rolling hash, winnowing fingerprints, TF-IDF with cosine similarity.
- Comparing every document pair directly is slow for large batches, so hashing and indexing make the comparison efficient.

Made with GAMMA

Literature Survey

- **String matching:** Rabin-Karp and related hashing find exact copied passages quickly.
- **Fingerprinting:** Winnowing, as used in MOSS, selects representative hashes from documents.
- **Vector similarity:** TF-IDF and cosine similarity catch reworded text.
- **Research gap:** Many tools only report a score. This project joins preprocessing, fingerprinting and passage-level reporting in one system.

Made with GAMMA

Problem Statement and Objectives

THE PROBLEM

- Manually checking submissions is slow, inconsistent and does not scale to large classes.
- Students disguise copying by changing words, reordering sentences and altering formatting.
- Simple exact-match checks miss paraphrased content, and pairwise comparison is costly at scale.

OBJECTIVE

Build an efficient, explainable detector that scores similarity and highlights matching passages between submissions.

Proposed Methodology / System Architecture

1. Document ingestion and text preprocessing layer
2. Shingling and fingerprint indexing layer
3. Similarity matching engine
4. Scoring, highlighting and reporting layer

Made with GAMMA

System Design / Workflow

1. Upload submissions (PDF, DOCX, TXT)
2. Extract and clean text: lowercase, strip punctuation, remove stop words
3. Generate k-gram shingles and hash them
4. Select fingerprints and store them in a hash index
5. Compare documents and compute Jaccard and cosine similarity
6. Report similarity scores and highlighted matches

Made with GAMMA

Final Implementation

- **Project setup:** [development environment and language]
- **Tools and technologies:** [libraries and frameworks used]
- **Folder structure:** [add folder layout]
- **Dataset:** [source of sample submissions or corpus]
- **Data format and preprocessing:** Text files and extracted documents, then cleaning and tokenization

Made with GAMMA

System and Algorithm Implementation

- **Main modules:** Text extraction, preprocessing, fingerprinting, matching, reporting.
- **Input processing:** Read, clean and tokenize each document.
- **Core algorithm:** Shingling, rolling hash, winnowing and similarity scoring.
- **Output:** Similarity percentages and highlighted matching text.

Made with GAMMA

Pseudocode

```
FUNCTION DetectPlagiarism(documents, k, threshold):
  for each doc in documents:
    tokens = Preprocess(doc)
    shingles = KGrams(tokens, k)
    fingerprints = Winnow(Hash(shingles))
    Index.Add(doc, fingerprints)
  for each pair (A, B) sharing fingerprints:
    score = Jaccard(A, B)
    if score >= threshold:
      record match (A, B, score)
  RETURN Report(matches)
```

Important Parameters

- **Shingle size (k):** Words per shingle; larger k means stricter matching.
- **Similarity threshold:** Score above which a pair is flagged.
- **Window size:** How many fingerprints winnowing keeps.
- **Stop-word handling:** Whether common words are removed before comparison.

Made with GAMMA

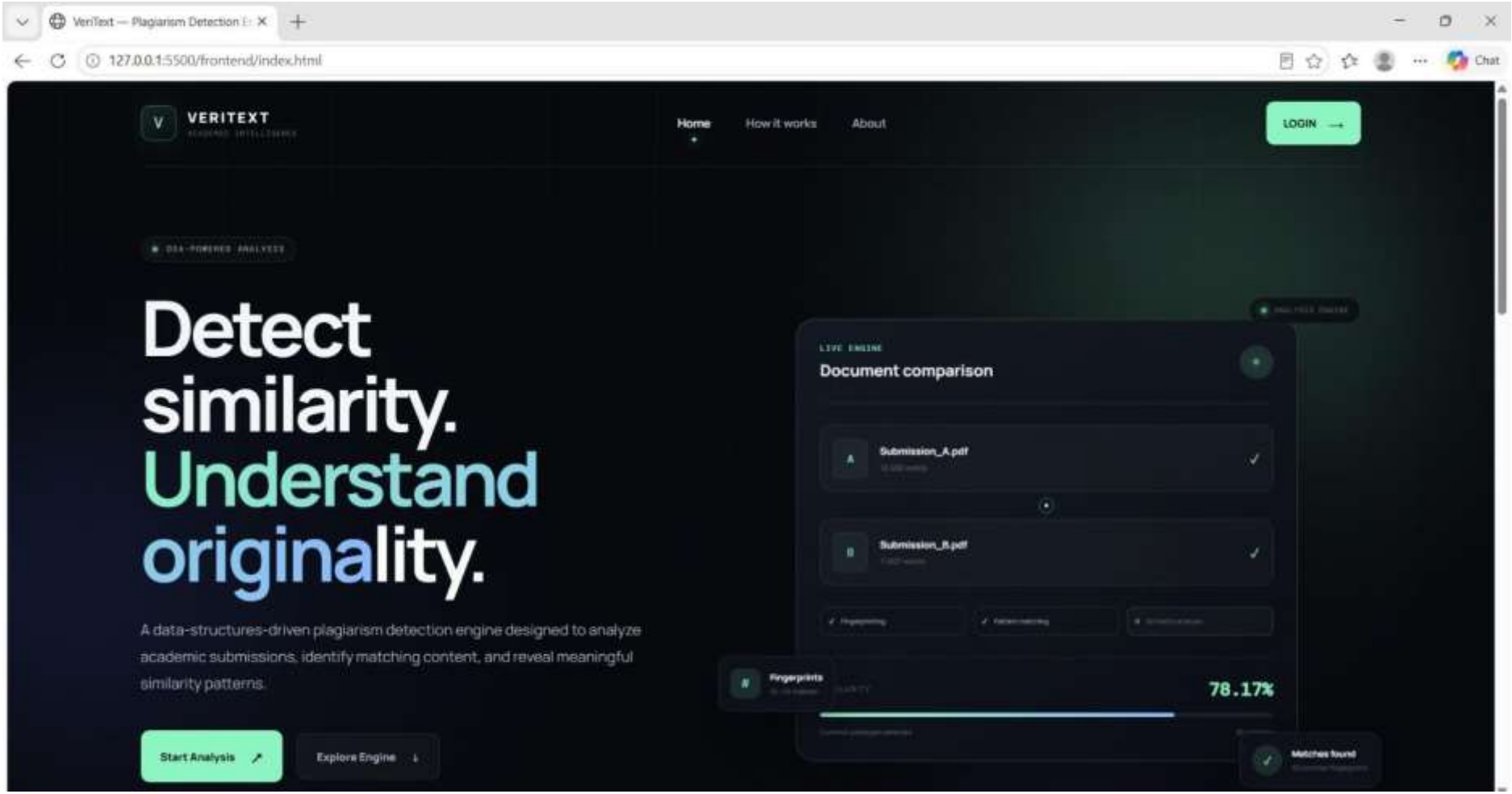
Algorithms and Performance Metrics

- **Algorithms:** k-gram shingling, Rabin-Karp rolling hash, winnowing, Jaccard and cosine similarity.
- **Similarity score:** Percentage of matched content per pair.
- **Precision and recall:** Accuracy of flagged cases against known copies.
- **Processing time:** Runtime as documents grow.

Made with GAMMA

User Interface: Main Screen

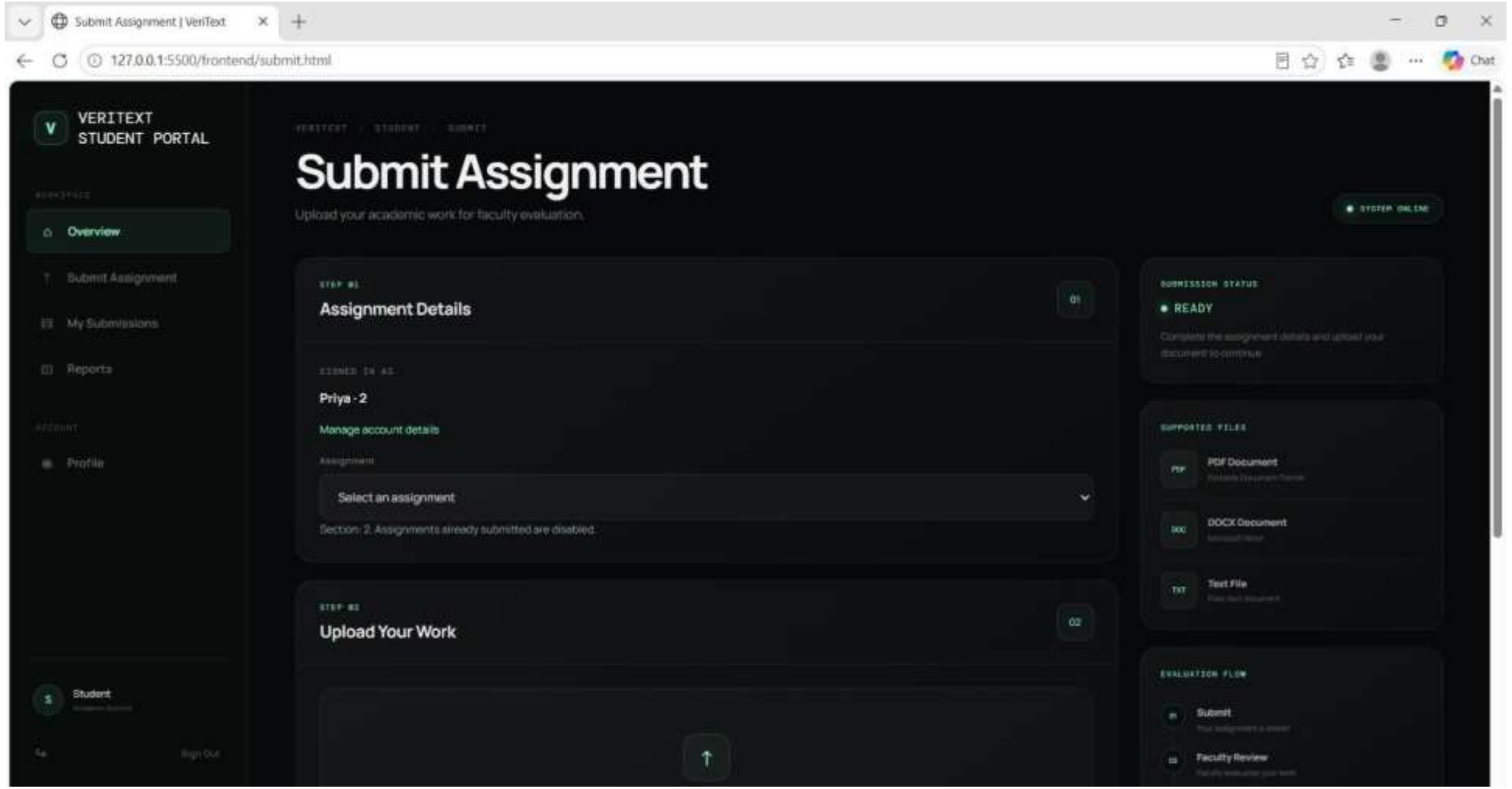
Add a screenshot here.



Made with GAMMA

User Interface: Input Screen

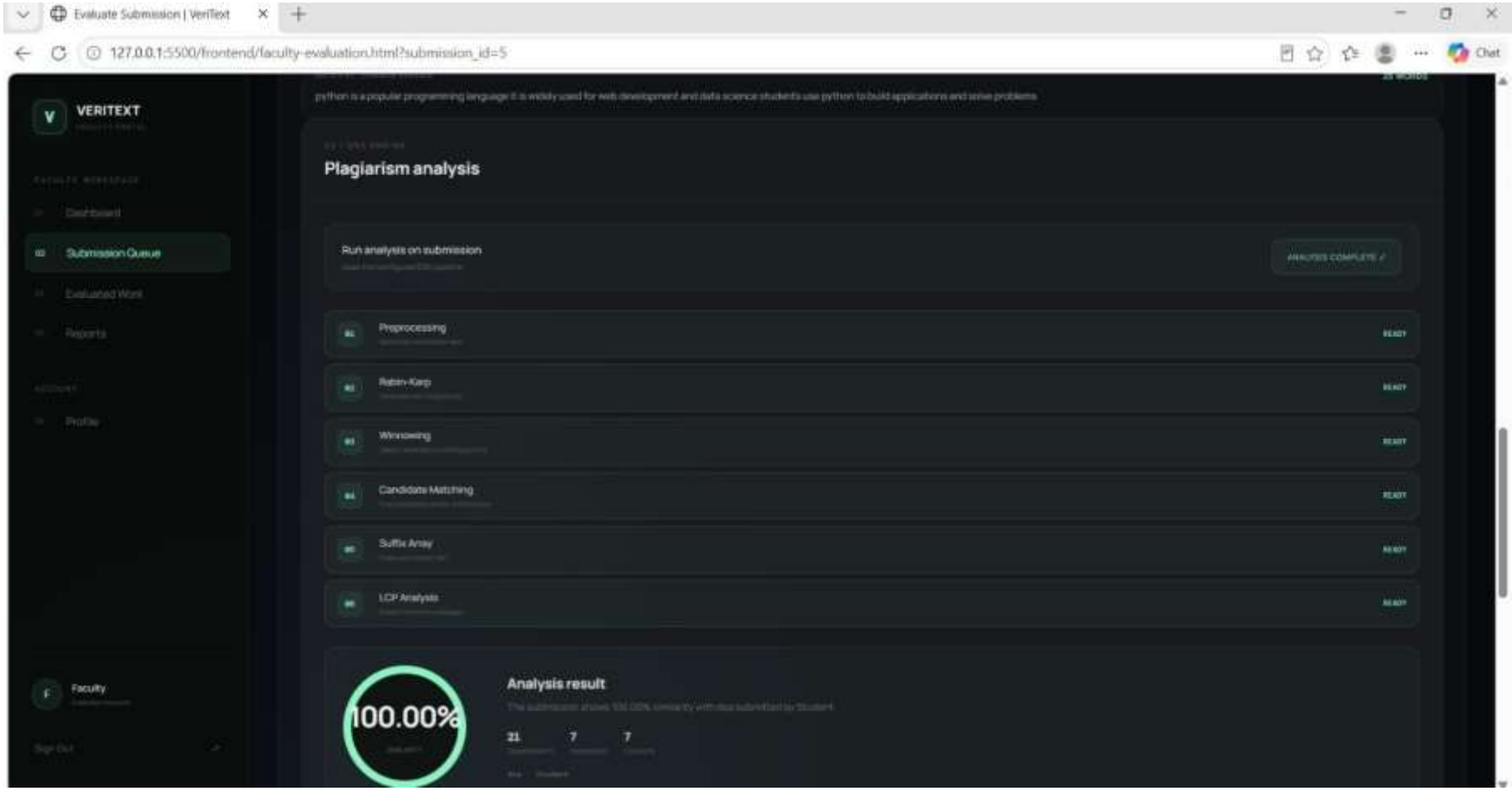
Placeholder



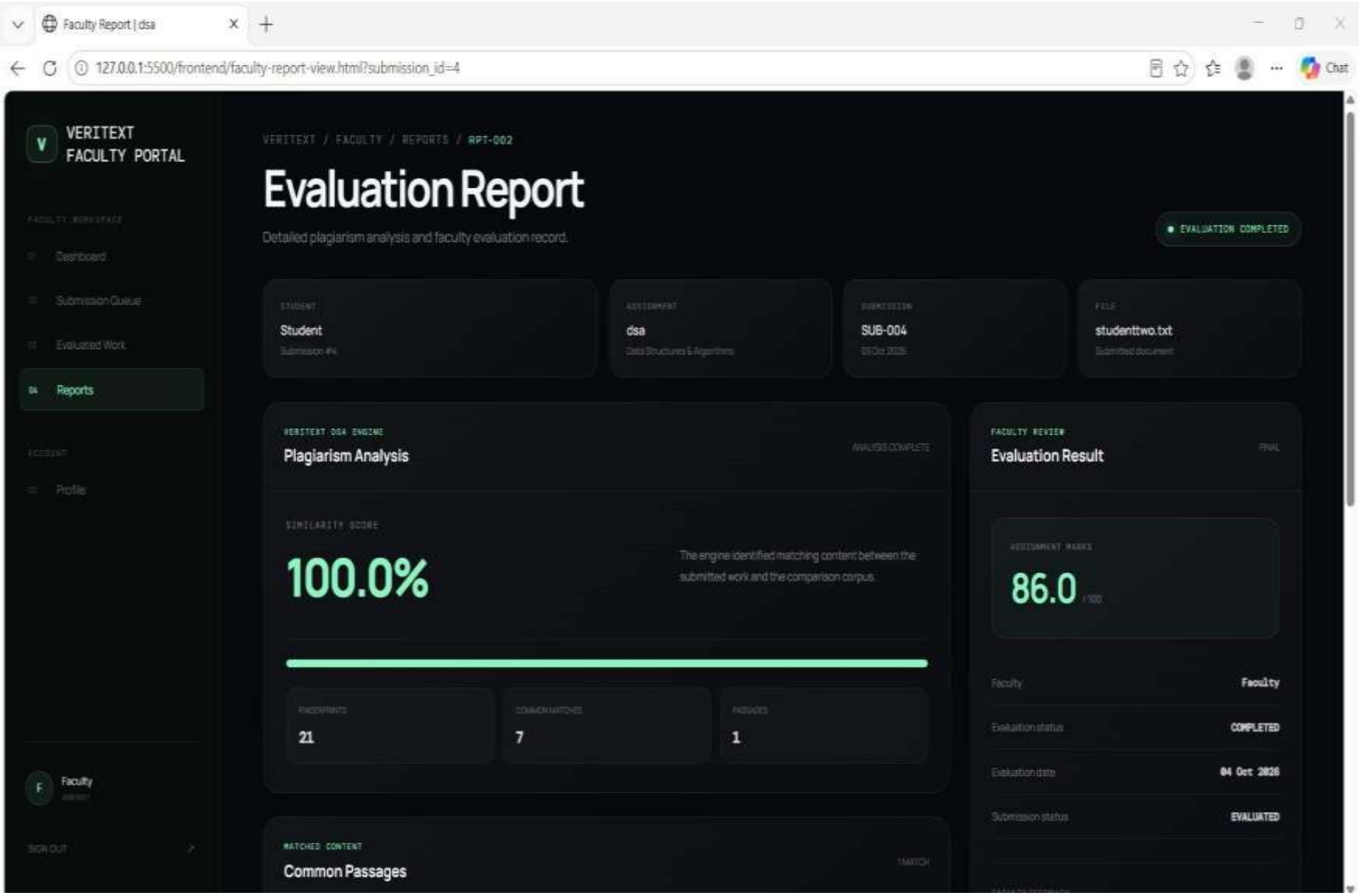
Made with GAMMA

User Interface: Processing Screen

Placeholder



Made with GAMMA



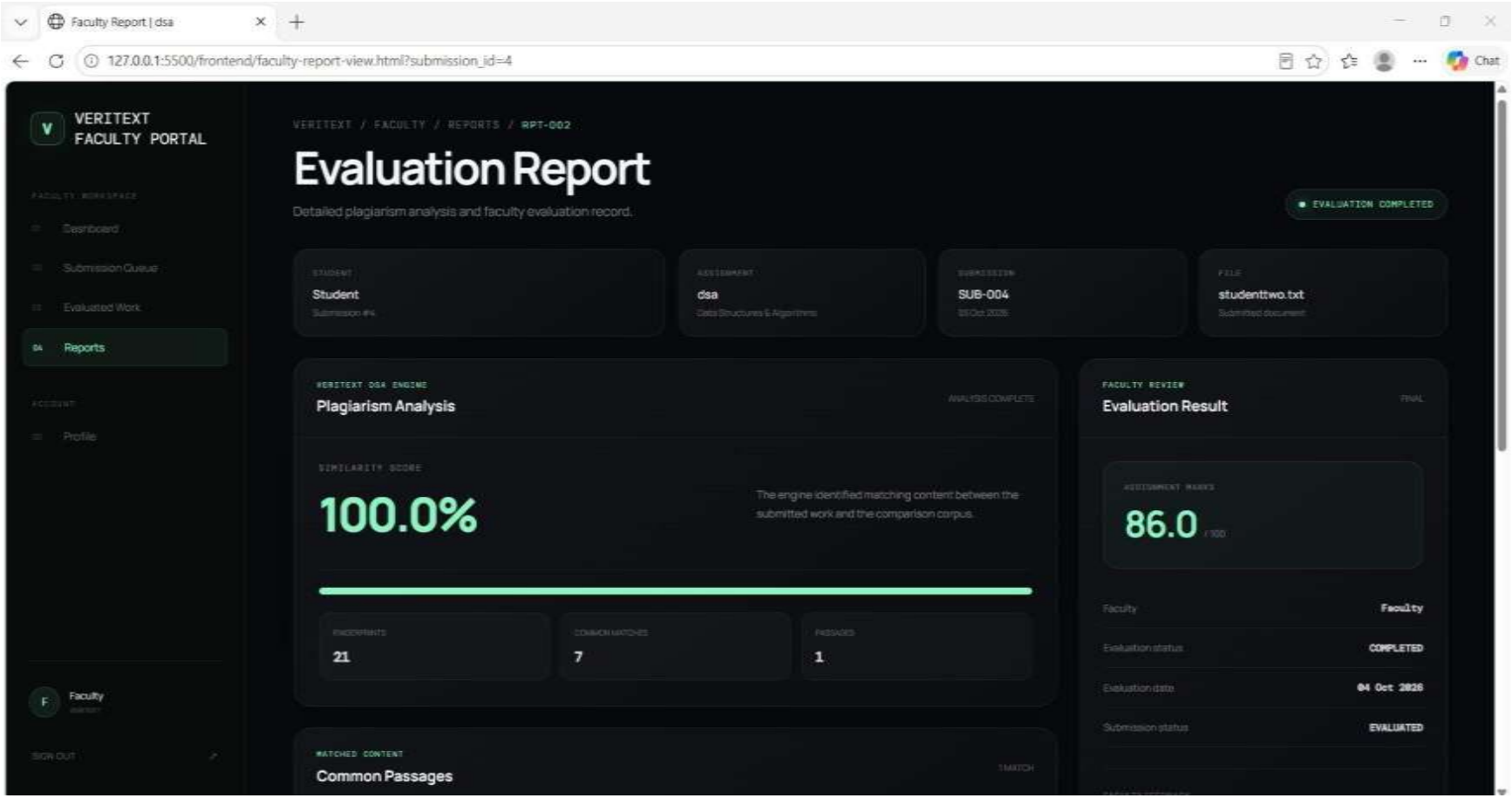
User Interface: Output Screen

Placeholder

Made with GAMMA

Testing: Expected Output

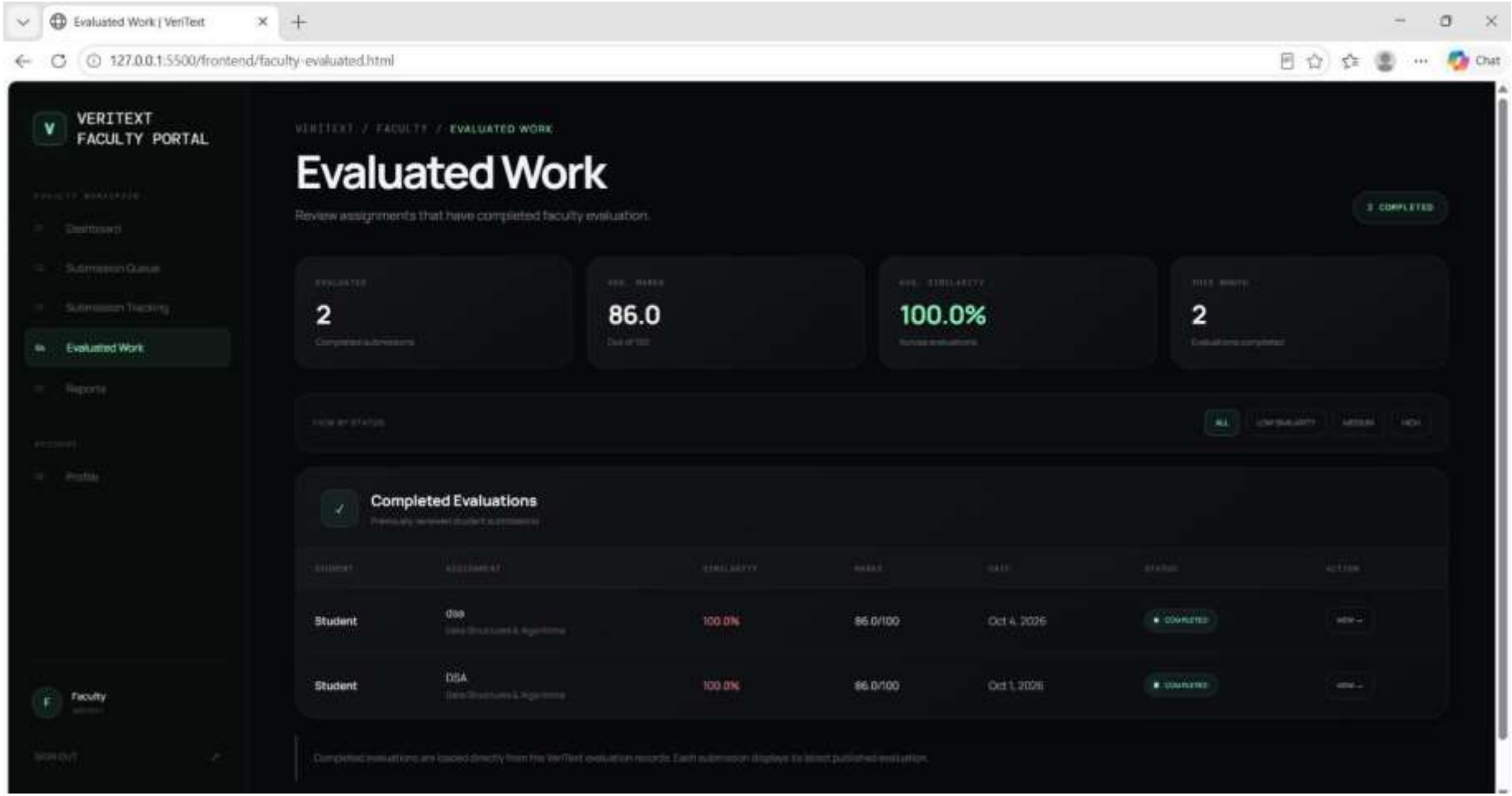
Placeholder



Made with GAMMA

Testing: Actual Output

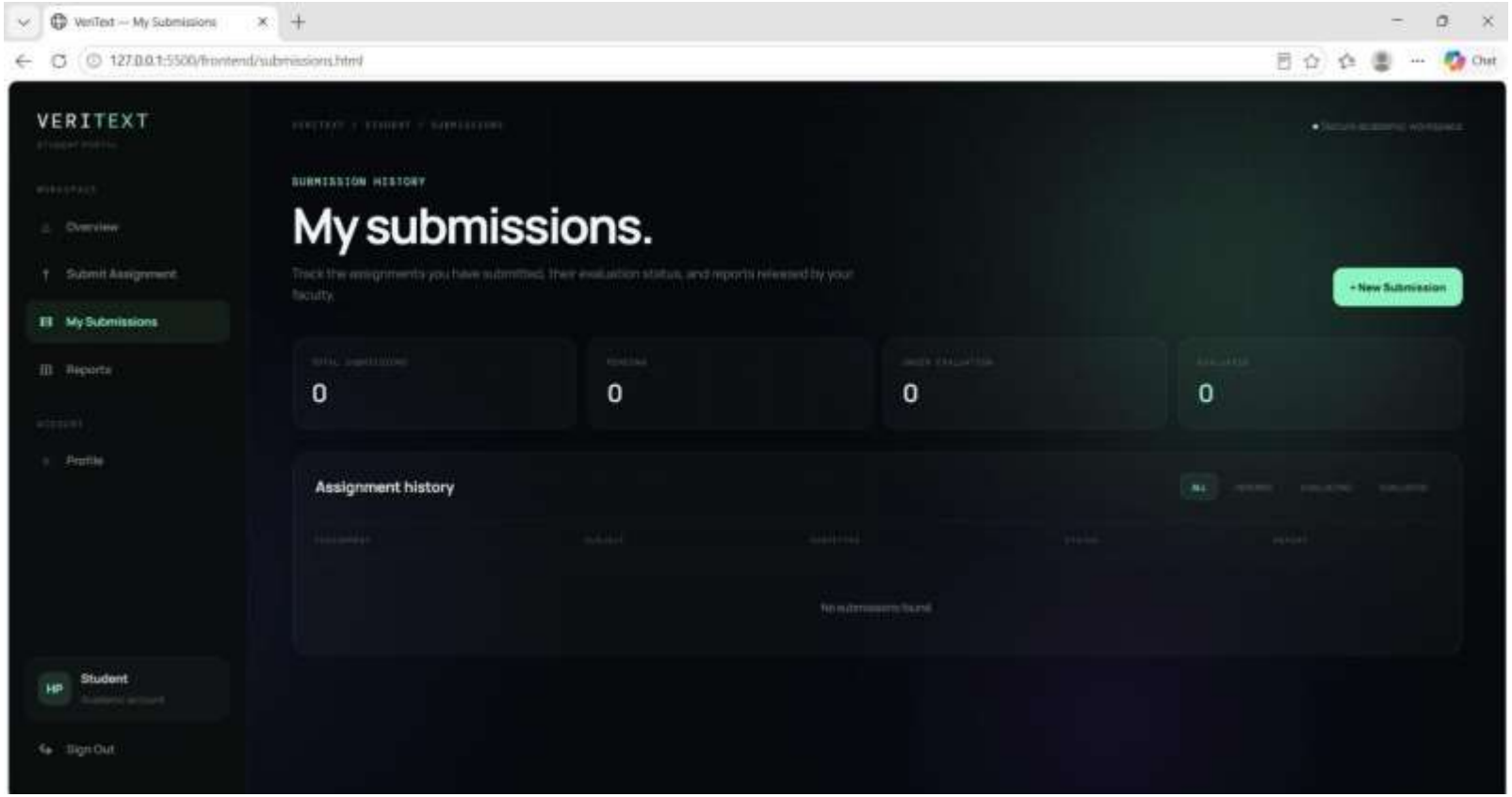
Placeholder



Made with GAMMA

Final Output: Output Screenshots

Placeholder



Made with GAMMA

Algorithm / Method Used

Algorithm:

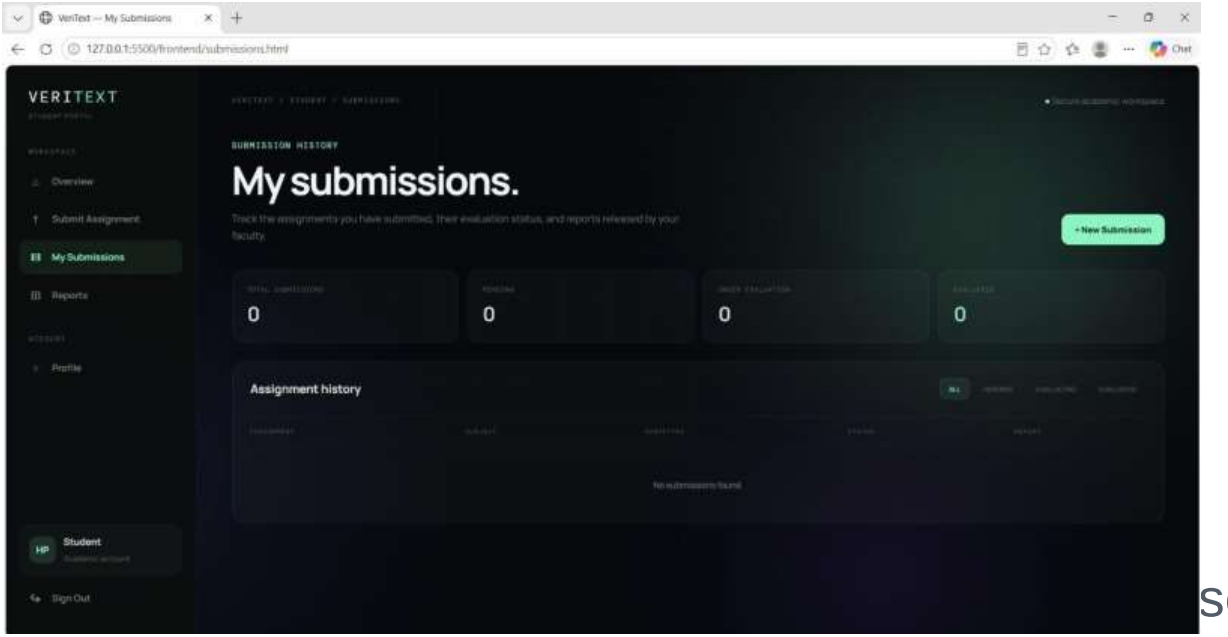
Rabin–Karp – generates text fingerprints using hashing.
Winnowing – selects important fingerprints for efficient comparison.
Suffix Array + LCP – identifies longer common passages.
Similarity Calculation – calculates the final similarity percentage.

Method:

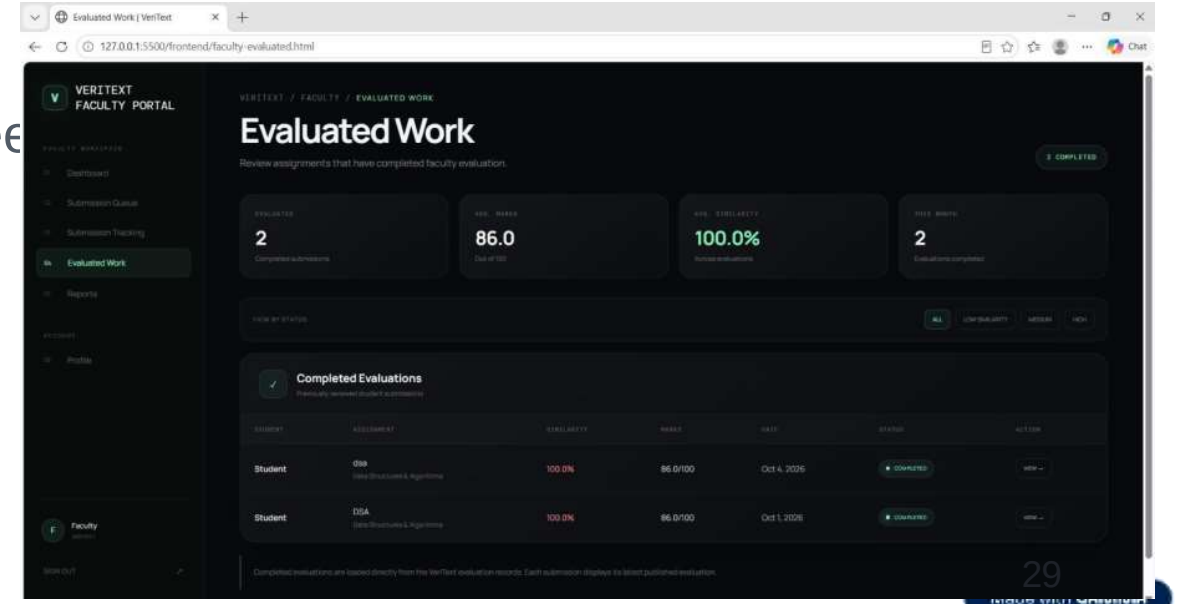
Upload → Preprocess → Fingerprinting → Winnowing
→ Compare → Find Common Passages → Similarity %
→ Faculty Evaluation → Final Report

Made with GAMMA

Experimental Results / Output Screenshots



scree



Performance Analysis

Fast fingerprint generation using Rabin–Karp hashing.
Winnowing reduces comparisons by selecting representative fingerprints.
Suffix Array + LCP efficiently identifies common passages.
Similarity percentage is generated along with matching content.
The system is suitable for comparing multiple academic submissions with reduced unnecessary processing.

Made with GAMMA

Discussion

Hashing and fingerprinting let the system flag copied passages far faster than comparing text word by word. Preprocessing and shingling make it robust to small edits, though heavy paraphrasing remains hard to catch. The tradeoff is between sensitivity and false positives, set by the shingle size and threshold.

Made with GAMMA

Conclusion

- The system detects copied content in academic submissions using fingerprinting and similarity measures.
- Preprocessing makes matching more reliable despite formatting changes.
- Highlighted matches make results explainable for reviewers.
- Hash-based indexing keeps comparison efficient across many documents.
- The approach is practical for universities and course instructors.

Made with GAMMA

Future Scope

- **Semantic detection:** use NLP embeddings to catch paraphrased text.
- **Source code plagiarism:** extend detection to programming assignments.
- **Web and database checks:** compare against online sources and past submissions.
- **Multilingual support:** detect translated plagiarism.
- **LMS integration:** plug into learning management systems.

Made with GAMMA

References

Add references here.

1. Schleimer, Wilkerson and Aiken (2003), "Winnowing: Local Algorithms for Document Fingerprinting".
2. Karp and Rabin (1987), Rabin-Karp string matching.

Made with GAMMA

Thank You

Questions and discussion

